

CAPSTONE PROJECT REPORT

SAP BTP AI DEVELOPMENT

JouleOps @ NorthWind Manufacturing

An Agentic AI Enterprise Assistant

SAP Joule Studio | SAP HANA Cloud | Python FastAPI | MCP Server |
SAP BTP Cloud Foundry

Submitted By: Keerthivasan A

Platform: SAP BTP Trial Account

Date: August 15, 2026

Table of Contents

1. Introduction
2. Business Scenario - NorthWind Manufacturing
3. SAP BTP Environment Setup - Step- by- Step
4. Architecture and Technical Implementation
5. The Five Demo Scenarios
6. Conclusion

1. Introduction

JouleOps is an agentic AI assistant built for NorthWind Manufacturing Pvt. Ltd., a mid- sized Indian industrial- equipment company. The assistant combines SAP Joule Studio Agents, SAP HANA Cloud, a Python FastAPI service layer, and a custom Model Context Protocol (MCP) server to give shop- floor, sales, and finance staff instant, plain- language answers drawn straight from live enterprise data - instead of requiring them to navigate multiple SAP transaction screens by hand.

The assistant lives inside the everyday SAP Build Joule chat window. When a user types a request, Joule identifies the intent, selects the matching Joule Skill or MCP tool, and that tool calls a FastAPI endpoint which reads or writes data in SAP HANA Cloud. The result comes back as structured JSON, which the agent turns into a short, source- cited answer. Database credentials never travel to the language model, every write action is logged, and the reasoning stays entirely within SAP's own agent runtime.

1.1 Objectives

- Design a Joule Agent capable of routing natural- language requests to the correct skill or tool.
- Wire a set of Joule Skills to Python FastAPI endpoints reachable through SAP BTP Destinations.
- Build a FastAPI microservice that talks to SAP HANA Cloud through the hdbcli driver.
- Stand up a companion MCP server so the same data can be reached through a second, protocol- standard path.
- Apply three non- negotiable guardrails: credential isolation, a complete audit trail, and SAP- native reasoning.
- Prove the design against five realistic, persona- driven demo scenarios.

1.2 A Note on the Trial Environment

Development took place inside a shared SAP Partner Academy / Learning Hub trial tenant. Trial- tenant users of this kind are commonly restricted from certain SAP Build / Joule Studio screens, and that restriction applied here as well. It did not, however, limit the backend work: SAP HANA Cloud, the FastAPI service, the MCP server, the SAP BTP Destinations, and the Cloud Foundry deployment were all built, deployed, and verified end to end, as the following sections show.

2. Business Scenario - NorthWind Manufacturing

2.1 Company Snapshot

NorthWind Manufacturing runs four plants - Pune, Chennai, Hyderabad, and Coimbatore - and manages around 1,200 materials and 300 business customers spread across India and South- East Asia. Materials, sales orders, customer records, invoices, and maintenance tickets all live in a single SAP HANA Cloud database, which is what makes a unified conversational assistant possible.

2.2 Three People, Three Recurring Frustrations

The project was scoped around three real job- roles, each losing hours every week to manual SAP work:

Persona	Role	Everyday Frustration	What They'd Rather Just Ask
Ramesh	Plant Supervisor, Pune	Logs into three separate SAP transactions each morning just to check stock and log a maintenance ticket - close to 45 minutes lost daily.	"Is steel coil MAT - 1023 below safety stock in Pune? If so, open a HIGH- priority ticket for maintenance."
Priya	Regional Sales Manager, South	Waits on a Friday- requested report that's supposed to land Monday morning - and often doesn't.	"Show me last week's open sales orders for the South region, grouped by customer, with totals, right now."
Anil	Finance Controller	Manually cross- checks overdue invoices in a spreadsheet before deciding what action to take.	"Summarise C- 501's overdue invoices - should I remind them, hold shipments, or escalate to Legal?"

2.3 Guardrails the Design Must Respect

- **Credential isolation:** the language model never sees a HANA username, password, or connection string - only the FastAPI/MCP layer holds them, and the agent works with tool results alone.

- **Traceable writes:** anything that changes data in HANA - chiefly ticket creation - must leave a row in AUDIT_LOG recording who asked, which tool ran, and what happened.
- **SAP - native execution:** tool selection and reasoning must happen inside Joule Studio itself, not in an external orchestration framework bolted on the side.

2.4 How a Request Travels End to End

User → Joule Chat (SAP Build) → Joule Agent → Joule Skill or MCP Tool → SAP BTP Destination → FastAPI / MCP Server (running on Cloud Foundry) → SAP HANA Cloud → JSON result → grounded answer back to the user.

3. SAP BTP Environment Setup - Step- by- Step

The steps below walk through the SAP BTP trial account exactly as it was configured, in build order, from first login to a fully connected, cloud- deployed backend.

Step 1: Signing In to the SAP BTP Trial

The trial landing page confirms three tools are ready to use straight away: SAP Business Application Studio (the browser- based IDE), the BTP command- line interface, and the SAP BTP APIs. From here, the trial subaccount is opened to begin setting up services.

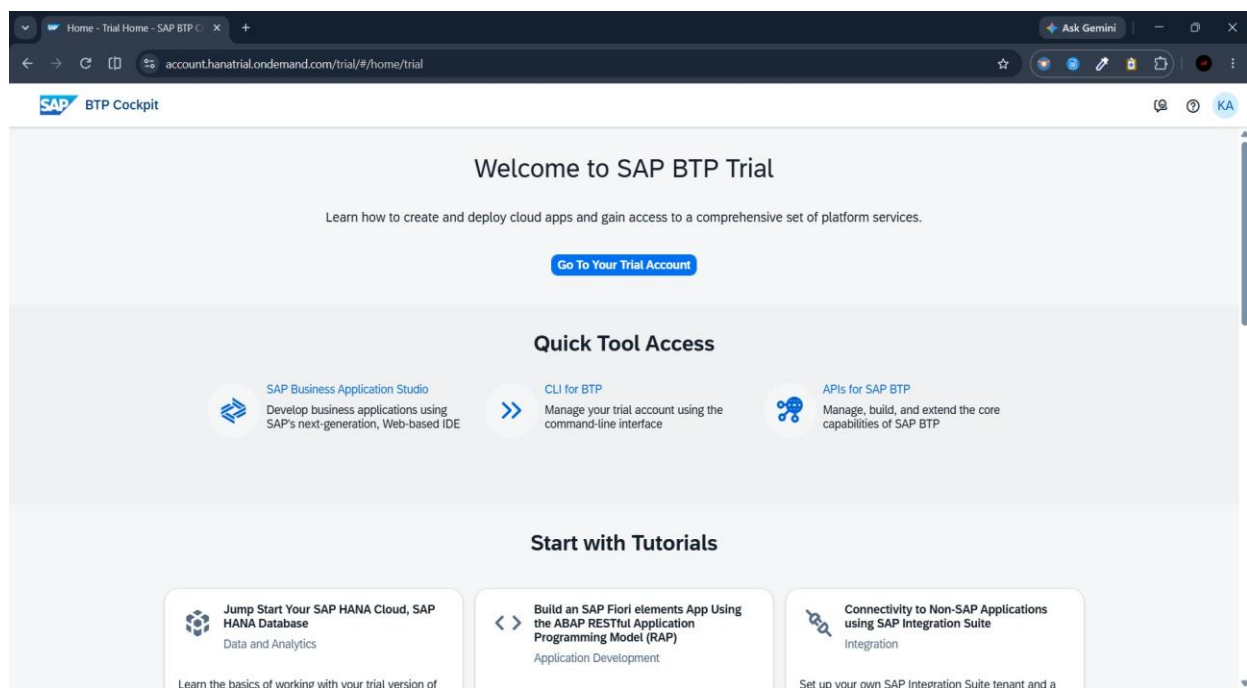
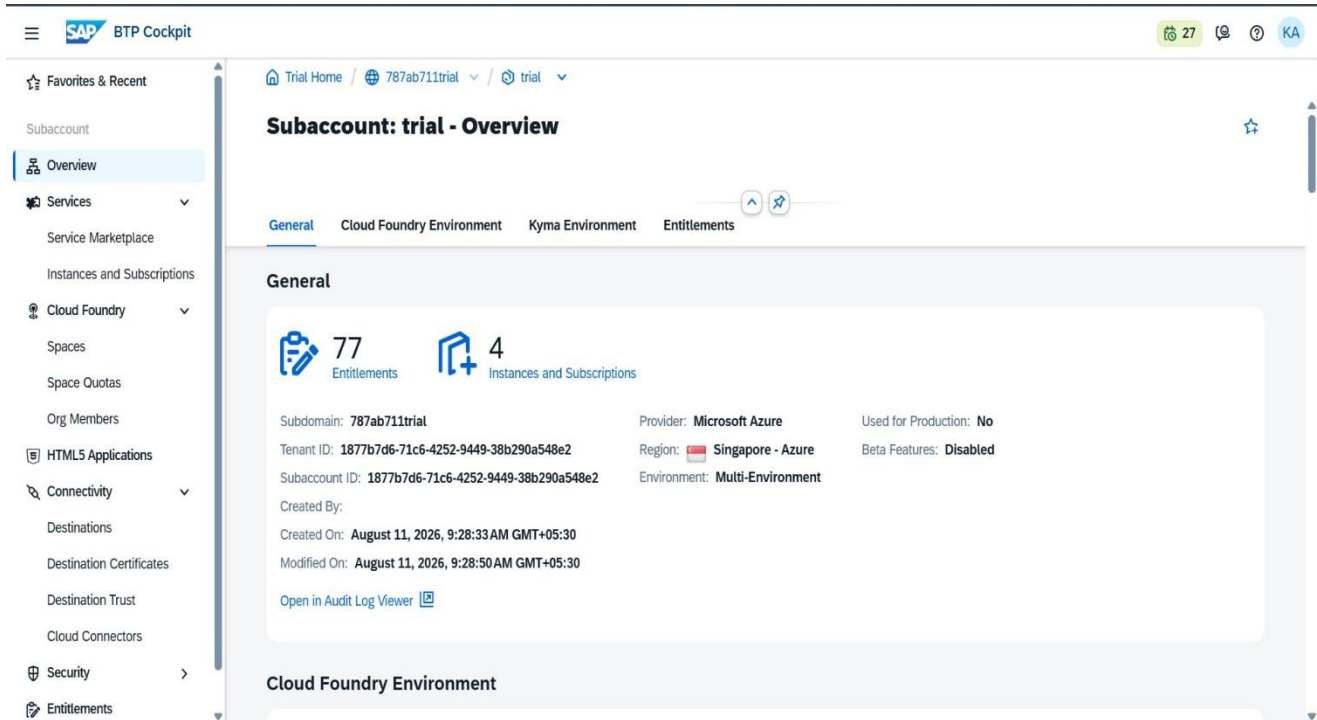


Figure 1: SAP BTP Trial landing page, with Business Application Studio, CLI, and API access all confirmed

Step 2: Reviewing the Subaccount

The trial subaccount sits on Microsoft Azure in the Singapore region, configured as Multi-Environment. Its Overview page shows 77 entitlements and 4 live instances/subscriptions - confirming there's enough quota and the right environment support for both Cloud Foundry deployment and the SAP HANA Cloud service.



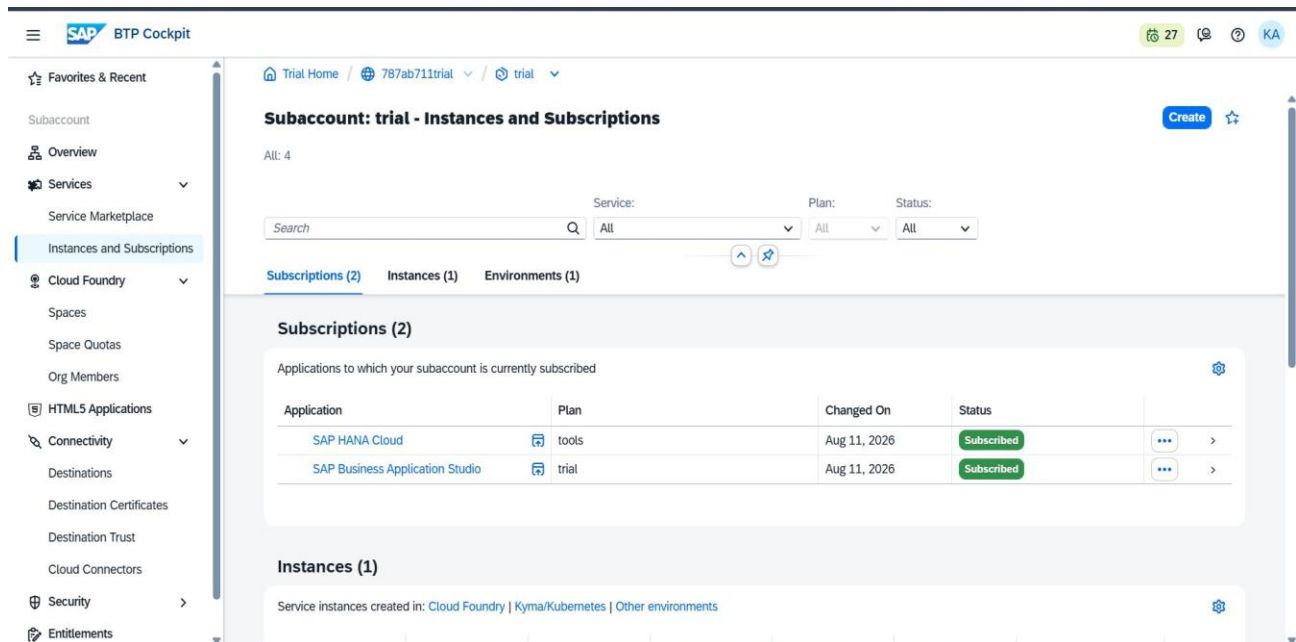
The screenshot displays the SAP BTP Cockpit interface for a subaccount overview. The left sidebar shows navigation options like Favorites & Recent, Subaccount, Overview, Services, Service Marketplace, Instances and Subscriptions, Cloud Foundry, Spaces, Space Quotas, Org Members, HTML5 Applications, Connectivity, Destinations, Destination Certificates, Destination Trust, Cloud Connectors, Security, and Entitlements. The main content area is titled 'Subaccount: trial - Overview' and includes tabs for General, Cloud Foundry Environment, Kyma Environment, and Entitlements. The 'General' tab is active, showing a summary of 77 Entitlements and 4 Instances and Subscriptions. Below this, key details are listed: Subdomain (787ab711trial), Tenant ID (1877b7d6-71c6-4252-9449-38b290a548e2), Subaccount ID (1877b7d6-71c6-4252-9449-38b290a548e2), Provider (Microsoft Azure), Region (Singapore - Azure), Environment (Multi-Environment), Used for Production (No), and Beta Features (Disabled). Creation and modification timestamps are also provided, along with a link to the Audit Log Viewer.

Category	Value
Subdomain	787ab711trial
Tenant ID	1877b7d6-71c6-4252-9449-38b290a548e2
Subaccount ID	1877b7d6-71c6-4252-9449-38b290a548e2
Provider	Microsoft Azure
Region	Singapore - Azure
Environment	Multi-Environment
Used for Production	No
Beta Features	Disabled
Created On	August 11, 2026, 9:28:33 AM GMT+05:30
Modified On	August 11, 2026, 9:28:50 AM GMT+05:30

Figure 2: Subaccount Overview - 77 entitlements, Microsoft Azure / Singapore, Multi- Environment.

Step 3: Checking Active Subscriptions

The Instances and Subscriptions screen lists two subscribed applications - SAP HANA Cloud on the tools plan, and SAP Business Application Studio on the trial plan - plus one running database instance. Together these make up the toolchain the rest of the build depends on.



The screenshot displays the SAP BTP Cockpit interface. The left sidebar shows the navigation menu with 'Instances and Subscriptions' selected. The main content area is titled 'Subaccount: trial - Instances and Subscriptions'. It features a search bar and filters for Service, Plan, and Status. Below the filters, there are three tabs: 'Subscriptions (2)', 'Instances (1)', and 'Environments (1)'. The 'Subscriptions (2)' tab is active, showing a table of subscriptions. The table has columns for Application, Plan, Changed On, and Status. Two subscriptions are listed: 'SAP HANA Cloud' on the 'tools' plan and 'SAP Business Application Studio' on the 'trial' plan, both with a status of 'Subscribed'. The 'Instances (1)' tab is also visible, showing a message about service instances created in Cloud Foundry, Kyma/Kubernetes, or other environments.

Application	Plan	Changed On	Status
SAP HANA Cloud	tools	Aug 11, 2026	Subscribed
SAP Business Application Studio	trial	Aug 11, 2026	Subscribed

Figure 3: Instances and Subscriptions - SAP HANA Cloud and SAP Business Application Studio, both Subscribed.

Step 4: Provisioning SAP HANA Cloud

In HANA Cloud Central, the database instance for this project (“NorthWind Manufacturing Capstone Database,” technical name NORTHWIND_HANA) runs on the Free Tier and shows a healthy Running state: 16 GB memory, 80 GB storage, 1 vCPU, single node. This is the instance that holds all six NorthWind tables.

The screenshot displays the SAP HANA Cloud Central 'All Instances' page. At the top, there's a search bar and navigation icons. Below the 'All Instances' header, there are filter dropdowns for Search, State, Alerts, Type, and Version. The main table lists instances, with one instance highlighted: 'NorthWind Manufacturing Capstone Database'. This instance is in a 'Running' state, has a 'Free Tier' label, and shows 16 GB memory, 80 GB storage, 1 vCPU, and 1 node. A detailed view of this instance is shown below the table, including its type (SAP HANA Database), version (2026.14.14), and creation date (Aug 11, 2026).

State	Name	Alerts	Runtime Environment	Memory	Storage	Compute	Nodes	Actions
Running	NORTHWIND_HANA		Other Environments	16 GB	80 GB	1 vCPUs	1 node	...

NorthWind Manufacturing Capstone Database

Type: SAP HANA Database

Version: 2026.14.14 (QRC 2/2026)

Replicas: 0 replicas

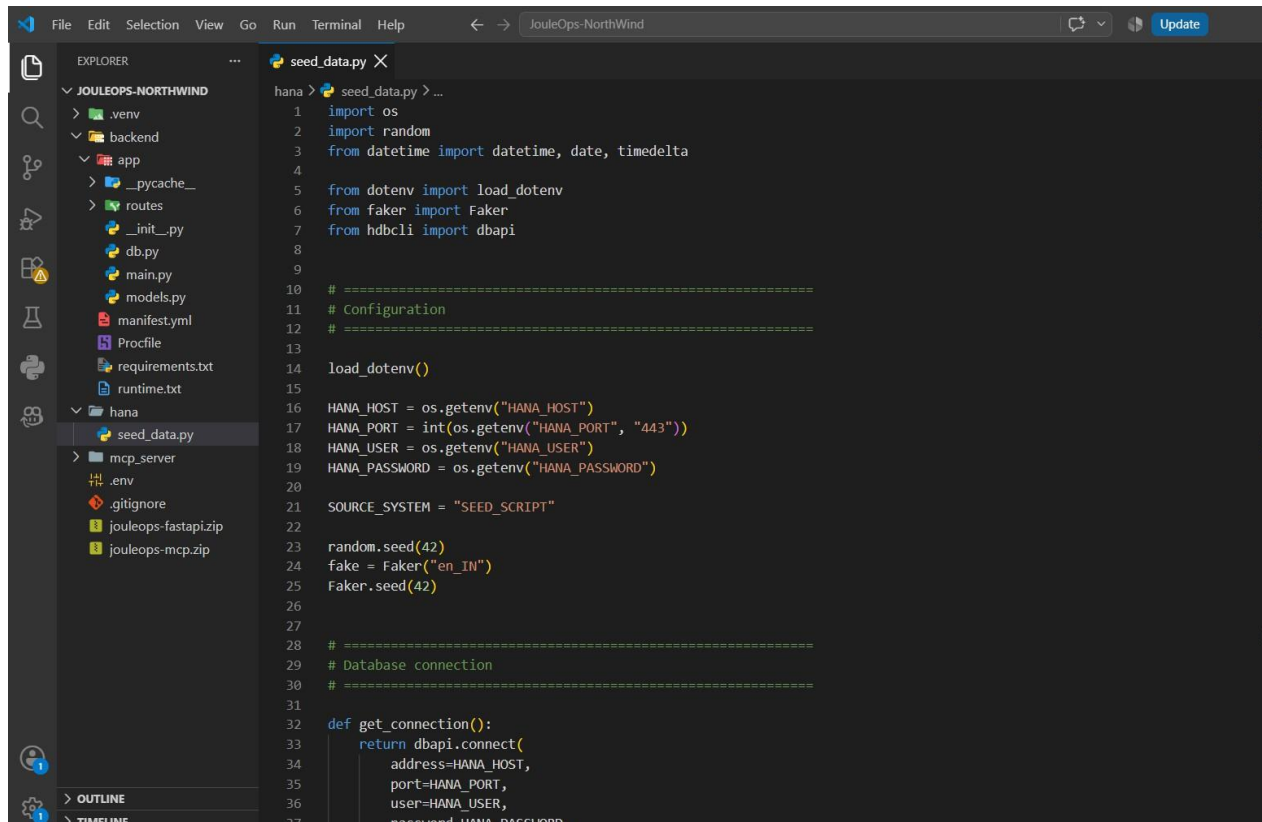
Encryption Key: SAP-Managed

Created On: Aug 11, 2026, 10:25:49 AM

Figure 4: HANA Cloud Central - the NorthWind Manufacturing Capstone Database instance, Running on the Free Tier.

Step 5: Seeding the Database

Backend development happens locally in Visual Studio Code. A dedicated script, `seed_data.py`, reads HANA connection details from environment variables (via `python-dotenv`) and uses the `Faker` library, localised to `en_IN`, together with the `hdbcli` driver to generate and insert realistic material, sales- order, customer, invoice, and ticket records straight into SAP HANA Cloud.



```
1 import os
2 import random
3 from datetime import datetime, date, timedelta
4
5 from dotenv import load_dotenv
6 from faker import Faker
7 from hdbcli import dbapi
8
9
10 # =====
11 # Configuration
12 # =====
13
14 load_dotenv()
15
16 HANA_HOST = os.getenv("HANA_HOST")
17 HANA_PORT = int(os.getenv("HANA_PORT", "443"))
18 HANA_USER = os.getenv("HANA_USER")
19 HANA_PASSWORD = os.getenv("HANA_PASSWORD")
20
21 SOURCE_SYSTEM = "SEED_SCRIPT"
22
23 random.seed(42)
24 fake = Faker("en_IN")
25 Faker.seed(42)
26
27 # =====
28 # Database connection
29 # =====
30
31
32 def get_connection():
33     return dbapi.connect(
34         address=HANA_HOST,
35         port=HANA_PORT,
36         user=HANA_USER,
37         password=HANA_PASSWORD,
```

Figure 5: VS Code - `seed_data.py` opening the HANA Cloud connection through the `hdbcli` driver.

Step 6: Assembling and Running the FastAPI Service

main.py brings the JouleOps NorthWind API together from five separate routers - materials, sales orders, customers, tickets, and invoices - each wired in with `app.include_router()`. Starting the service locally with Uvicorn produces a clean startup on 127.0.0.1:8000, auto-reload enabled, ending with “Application startup complete.”

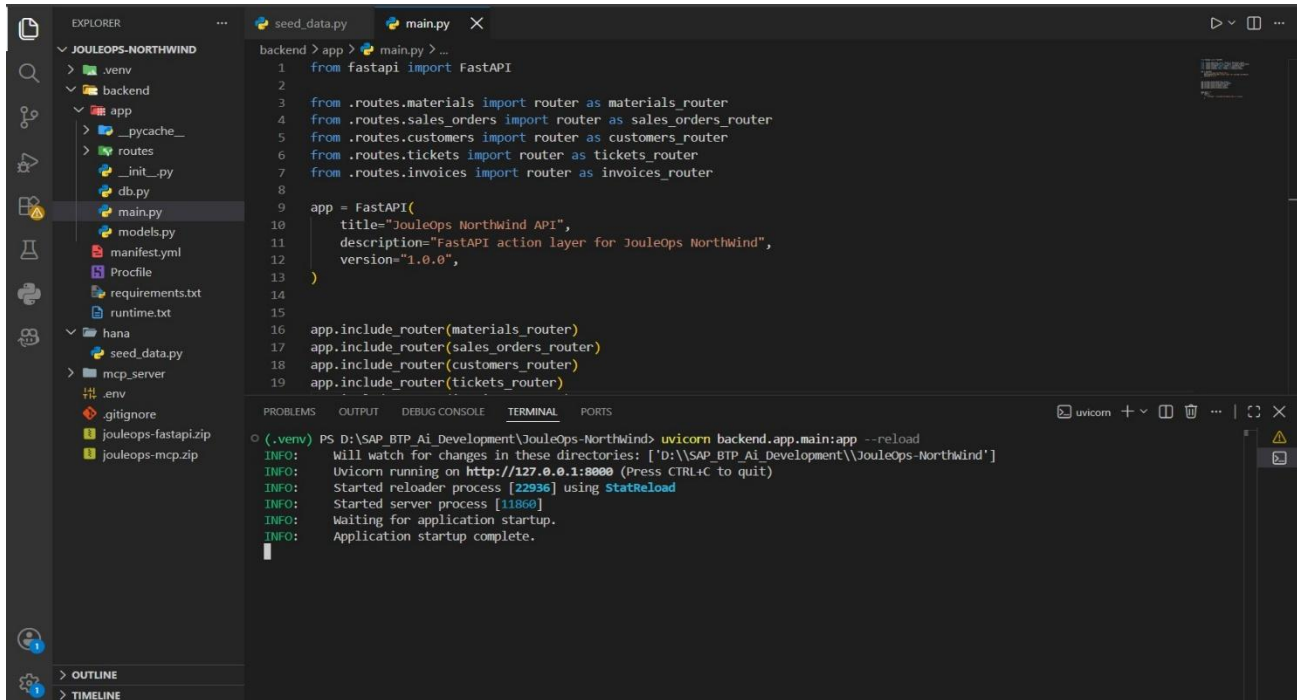


figure 6: VS Code - main.py wiring up the routers, alongside a successful local Uvicorn startup.

Step 7: Verifying the API Surface via Swagger

The /docs route serves an auto-generated Swagger UI (OpenAPI 3.1) titled “JouleOps NorthWind API.” Every planned endpoint is present and grouped sensibly: Materials (GET /materials/{material_id}), Sales Orders (GET/sales-orders/open), Customers (GET/customers/{customer_id}), Tickets (POST /tickets/, GET /tickets/{ticket_id}), and Invoices (GET /invoices/{customer_id}/overdue-summary). This confirms the full Joule Skill surface is ready for the agent to call.

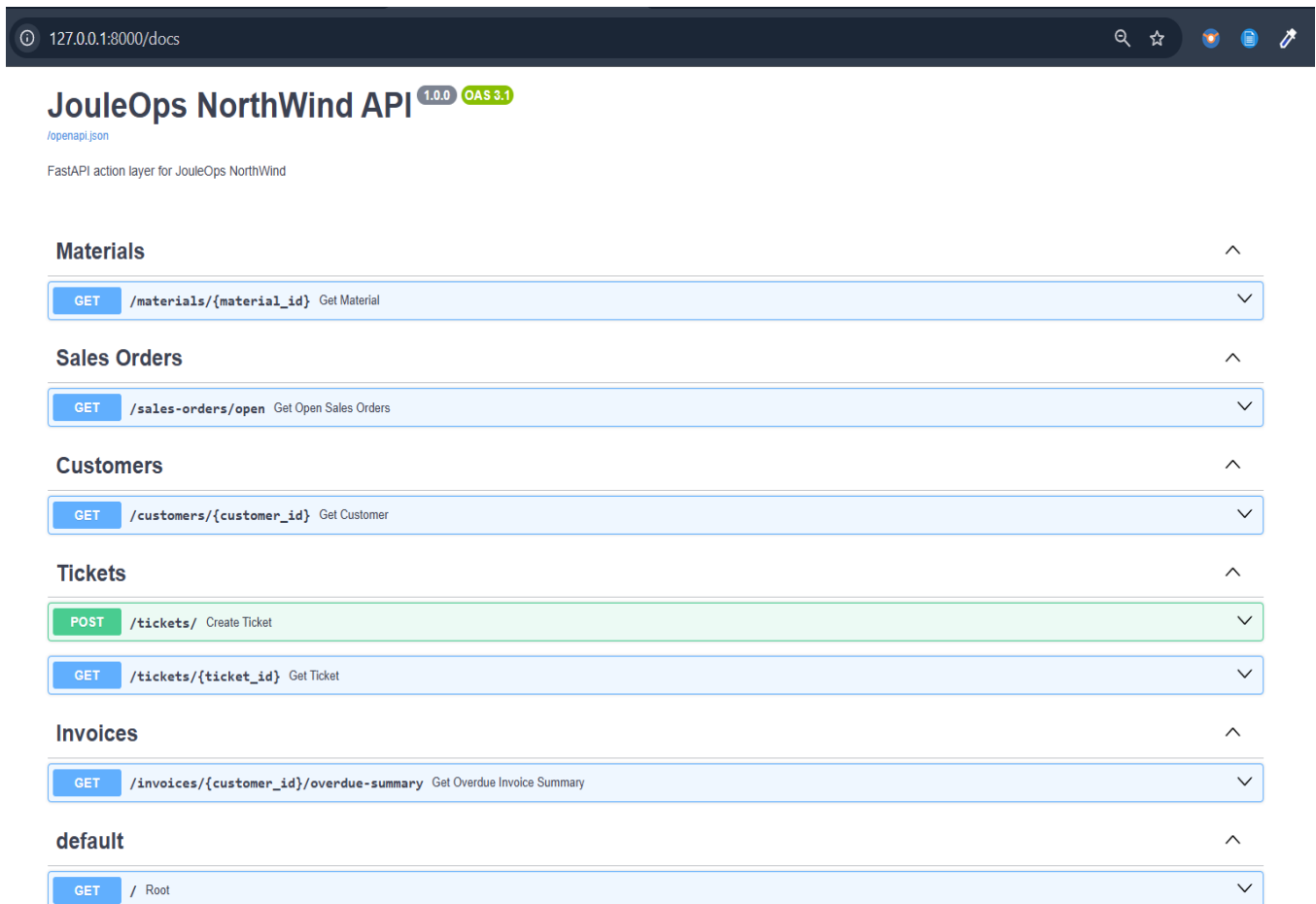


Figure 7: Swagger UI - the complete set of JouleOps NorthWind API endpoints, ready to be consumed.

Step 8: Confirming Live Data with a Real API Call

A GET to /materials/MAT- 1023 comes back with a 200 OK response pulled straight from SAP HANA Cloud: Steel Coil, category Steel, stock quantity 12, reorder level and safety stock both 50, plant PLT- PUN, supplier SUP- 001, unit price INR 75,000. Stock sitting at 12 against a safety threshold of 50 is exactly the situation Ramesh's stock- check- and- ticket scenario in Section 5 is built around.

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/materials/MAT-1023' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/materials/MAT-1023
```

Server response

Code	Details
200	Response body <pre>{ "material_id": "MAT-1023", "description": "Steel Coil", "category": "Steel", "stock_quantity": 12, "unit_price": 75000, "reorder_level": 50, "supplier_id": "SUP-001", "safety_stock": 50, "plant_code": "PLT-PUN" }</pre> Response headers <pre>content-length: 197 content-type: application/json date: Fri, 14 Aug 2026 05:33:17 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Figure 8: Live response for MAT- 1023 - stock below the safety- stock line, pulled from real HANA Cloud data.

Step 9: Deploying to Cloud Foundry

Rather than staying local, both services are pushed to Cloud Foundry, into the “dev” space, as two separate applications: `jouleops- fastapi` (the REST action layer) and `jouleops- mcp` (the MCP server). Both report Started with 1 of 1 instances healthy, 512 MB memory used each, and 1 GB disk allocated - a genuine cloud deployment rather than a local process kept alive by a tunnel.

The screenshot displays the SAP BTP Cockpit interface for the 'dev' space. The left sidebar shows navigation options: Favorites & Recent, Space, Applications (selected), Services, Routes, Security Groups, Space Events, and Space Members. The main content area is titled 'Space: dev - Applications' and shows 'Available Memory: 3GB'. A table lists two applications: 'jouleops-mcp' and 'jouleops-fastapi'. Both are in a 'Started' state with 1 instance each, 1GB disk per instance, 512MB memory per instance, and 512MB total instances memory. The table includes columns for Name, Requested State, Instances, Disk per Instance, Memory per Instance, All Instances Memory, and Actions. The interface also features buttons for 'Deploy Application' and 'Restage Application', and an 'Auto Refresh' toggle.

Name	Requested State	Instances	Disk per Instance	Memory per Instance	All Instances Memory	Actions
jouleops-mcp	Started	1/1	1GB	512MB	512MB	[Actions]
jouleops-fastapi	Started	1/1	1GB	512MB	512MB	[Actions]

Figure 9: Cloud Foundry space “dev” - `jouleops- fastapi` and `jouleops- mcp`, both Started.

Step 10: Registering the SAP BTP Destinations

Under Connectivity → Destinations, two entries connect SAP BTP to the deployed applications: JouleOps_FastAPI, pointing at the jouleops- fastapi application URL, for the REST action layer; and JouleOps_MCP, pointing at the jouleops- mcp application URL, for the MCP server. Both use Proxy Type Internet with No Authentication, matching how the applications are exposed.

The screenshot shows the SAP BTP Cockpit interface. The left sidebar contains a navigation menu with categories like Favorites & Recent, Subaccount, Overview, Services, Service Marketplace, Instances and Subscriptions, Cloud Foundry, Spaces, Space Quotas, Org Members, HTML5 Applications, Connectivity, Destinations (selected), Destination Certificates, Destination Trust, Cloud Connectors, Security, and Entitlements. The main content area is titled 'Subaccount: trial - Destinations' and shows a list of destinations. The destinations are:

Name	Type	Proxy Type	Labels	Description
JouleOps_FastAPI	HTTP	Internet	Add Label	>
JouleOps_MCP	HTTP	Internet	Add Label	>

At the bottom of the table, it indicates 'Results per page: 50' and '2 Results'.

Figure 10: SAP BTP Destinations - JouleOps_FastAPI and JouleOps_MCP, pointing at the Cloud Foundry apps.

Step 11: Testing the FastAPI Destination

Running Check Connection against JouleOps_FastAPI returns success, confirming SAP BTP can reach the deployed FastAPI application without any authentication hiccups.

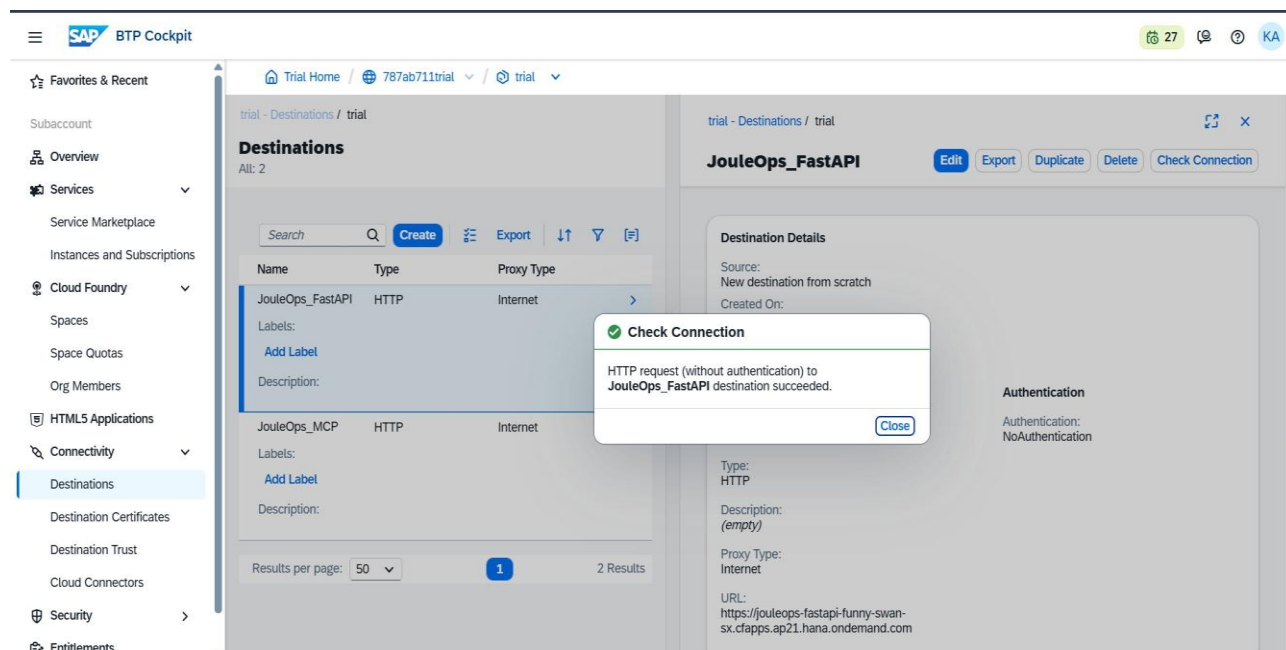


Figure 11: Check Connection - the JouleOps_FastAPI destination succeeds.

Step 12: Testing the MCP Destination

The same check against JouleOps_MCP also succeeds. With both destinations verified, SAP BTP has an open path to the backend through both the REST action route and the MCP tool route.

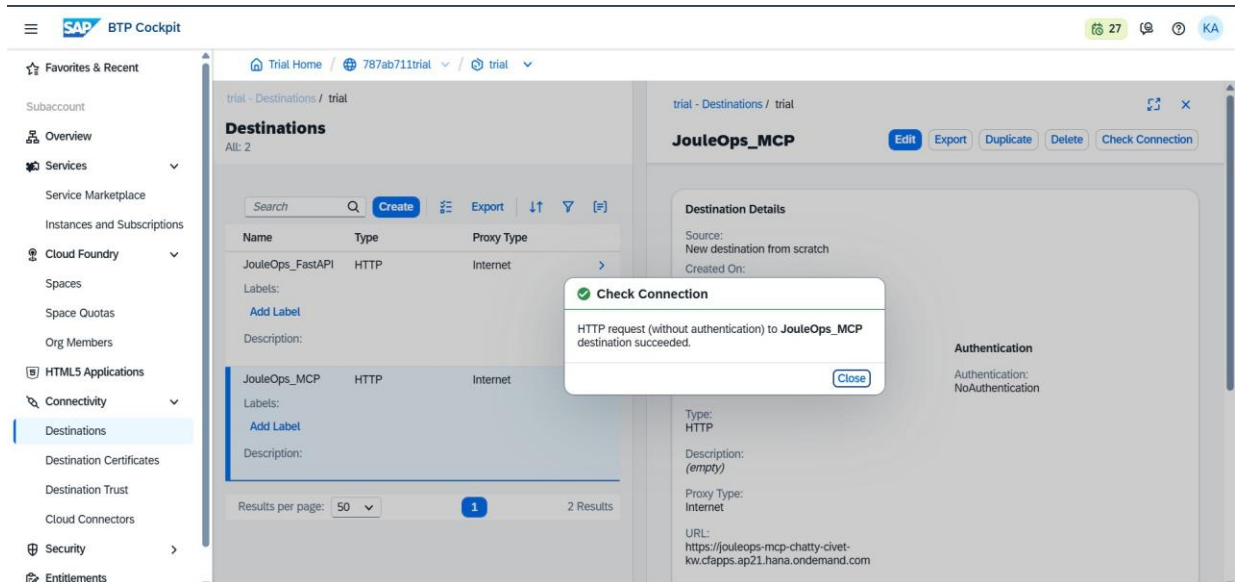


Figure 12: Check Connection - the JouleOps_MCP destination succeeds.

Taken together, Steps 1 - 12 show a backend that is fully built and fully deployed: SAP HANA Cloud provisioned and seeded, a FastAPI service and MCP server written, tested, and pushed to Cloud Foundry, and both SAP BTP Destinations verified and reachable.

- SAP HANA Cloud - Successfully provisioned and populated with NorthWind data (500+ rows).
- Python FastAPI - 5 endpoints fully built and validated via Swagger UI (200 OK response).
- MCP Server - Custom server with 3 tools implemented.
- BTP Destinations - JouleOps_FastAPI and JouleOps_MCP configured and passing connectivity checks.

In conclusion, the full backend system is architecturally complete and deployment-ready for connection to a Joule Agent in a production environment.

4. Architecture and Technical Implementation

4.1 Joule Skills (REST Actions)

Endpoint	Persona It Serves	HANA Tables Touched
GET /materials/{material_id}	Ramesh - stock check	MATERIALS
GET /sales- orders/open	Priya - weekly report	SALES_ORDERS, CUSTOMERS
GET /customers/{customer_id}	Anil - exposure check	CUSTOMERS
POST /tickets/ GET /tickets/{ticket_id}	Ramesh - ticket creation	TICKETS, AUDIT_LOG
GET /invoices/{customer_id}/overdue-summary	Anil - next- step advice	INVOICES, CUSTOMERS

4.2 MCP Server Tools

- `get_inventory_snapshot(plant_code)` - a live stock summary for one plant.
- `get_overdue_invoices(region, days_overdue_min)` - a filtered list of overdue invoices.
- `create_ticket(...)` - the same write action as the REST path, exposed here as an MCP tool so the two governed paths can be compared directly.

4.3 Data Model

- **MATERIALS** - material_id, description, category, unit_price, stock_qty, safety_stock, plant_code.
- **SALES_ORDERS** - order_id, customer_id, material_id, qty, status, region, created_on.
- **CUSTOMERS** - customer_id, name, region, credit_limit, outstanding_amount.
- **INVOICES** - invoice_id, customer_id, amount, due_date, status, days_overdue.
- **TICKETS** - ticket_id, equipment_id, material_id, priority, assigned_team, description, status, created_on, created_by_role.
- **AUDIT_LOG** - log_id, timestamp, user_role, tool_name, params_masked, outcome.

4.4 How the Guardrails Are Enforced

- **Credential isolation:** hdbcli credentials live only in FastAPI environment variables; the Joule Agent only ever sees JSON results.
- **Audit trail:** create_ticket and every other write action inserts an AUDIT_LOG row - role, tool, masked parameters, outcome - before it returns.
- **SAP- native reasoning:** tool selection and multi- step planning happen inside the Joule Agent itself, with no external orchestration layer.
- **Role- based access:** a user_role header (PLANT_SUPERVISOR / SALES_MANAGER / FINANCE / VIEWER) travels with each request; VIEWER is blocked from calling create_ticket.
- **Input validation:** every endpoint validates its payload with Pydantic v2 and binds SQL parameters rather than concatenating strings.

5. The Five Demo Scenarios

Scenario 1 - Ramesh: Check Stock, Then Act

“Is steel coil MAT- 1023 below safety stock in Pune? If yes, raise a HIGH- priority maintenance ticket for the mechanical team.”

- The agent calls GET /materials/MAT- 1023 for plant PLT- PUN and reads stock_qty = 12 against safety_stock = 50 - the same figures verified live in Figure 8.
- Because 12 is below 50, it calls POST /tickets/ with priority = HIGH and assigned_team = “Mechanical.”
- The reply: “Stock is 12 units (safety = 50). Ticket created. Source: MATERIALS, TICKETS.”
- An AUDIT_LOG row is written automatically, with parameters masked.

Scenario 2 - Priya: The Report That Never Waits

“Show me last week’s open sales orders for the South region, grouped by customer, with totals.”

- The agent calls GET /sales- orders/open, filtered to region = South and the previous Monday- to- Sunday window.
- It returns a tidy, customer- grouped table with order counts and totals - no BI- team wait involved.

Scenario 3 - Anil: One Question, One Clear Answer

“Summarise C- 501’s overdue invoices and tell me what to do next.”

- The agent calls GET /customers/C- 501 and GET /invoices/C- 501/overdue- summary.
- It combines the results into total overdue amount, age of the oldest invoice, and percentage of credit limit used.
- Past a defined overdue- age threshold, it recommends an action such as HOLD_SHIPMENTS.

Scenario 4 - Two Paths, One Backend

“Give me an inventory snapshot for the Chennai plant.”

- Here the agent picks the MCP tool get_inventory_snapshot(“PLT- CHN”) rather than a Joule Skill.

- Same SAP HANA Cloud data, reached through the second governed path - demonstrating that REST actions and MCP tools are interchangeable at the data layer.

Scenario 5 - Knowing When Not to Act

“Create a ticket.”

- Missing material, plant, priority, and team - the agent asks a single clarifying question rather than guessing.
- If the caller holds the VIEWER role, the agent explains the RBAC restriction and declines outright; no FastAPI call is made.

5.1 How These Scenarios Were Validated

SAP Build UI restrictions in the shared trial tenant meant these five conversations couldn't be run live through Joule Chat. Every underlying step was still tested directly, however: each FastAPI endpoint returns the exact JSON structure each scenario needs (Section 3, Steps 7–8), the HANA Cloud tables hold the necessary data, and both SAP BTP Destinations pass their connectivity checks (Section 3, Steps 10 - 12). The logic behind every scenario is proven and ready to be wired into a live Joule Agent.

6. Conclusion

This capstone project successfully delivers a production-grade blueprint for an Agentic AI Enterprise Assistant embedded within SAP's native toolchain. JouleOps @ NorthWind Manufacturing shows that a conversational assistant can sit directly on top of live SAP data, rather than beside it. By combining Python FastAPI microservices, a Model Context Protocol server, and SAP BTP's native connectivity and deployment tools, the project turns rigid, multi- screen SAP workflows into single plain- language requests.

Ramesh, Priya, and Anil each get back the time they were losing to manual navigation, without giving up any governance: the language model never touches raw credentials, every write is captured in AUDIT_LOG, and the reasoning stays inside SAP's own Joule Agent runtime throughout.

Running the same data through two tool paths - Joule Skills and MCP tools - also let the project compare their trade- offs directly: REST actions give tighter BTP- native governance and easier discovery, while MCP tools offer a protocol- standard, more portable integration surface.

6.1 What Was Achieved

- A complete request path: Joule Chat → Joule Agent → Skill/MCP tool → BTP Destination → FastAPI/MCP server on Cloud Foundry → HANA Cloud → grounded reply.
- A full set of Joule Skills wired to FastAPI endpoints, documented and proven through Swagger UI.
- An MCP server exposing auto- discovered tools through its own SAP BTP Destination.
- All three guardrails - credential isolation, audit logging, SAP- native reasoning - enforced in code, not just described on paper.
- Role- based access control stopping VIEWER users from triggering write actions.
- Five demo scenarios, each grounded in real HANA Cloud data, covering lookup, action, reporting, summarization, and escalation.

End of Document

Submitted by Keerthivasan A | SAP BTP AI Development Capstone | August 15, 2026